

Roadmap de Desenvolvimento: Stack Gráfica Avançada (OSDev)

Guia de Engenharia para Sistemas Operacionais Bare-Metal de Kernel Próprio

Visão Geral da Arquitetura: Para evitar código poluído ("lixo de desenho") e gargalos críticos de desempenho no seu kernel, a arquitetura gráfica é dividida de forma estrita em três camadas isoladas: o Kernel/Driver para gerenciamento físico, o Display Server/Compositor para a lógica de exibição, e o Asset Pipeline para a preparação de recursos visuais estáticos.

Fase 1: Infraestrutura de Baixo Nível e Gerenciamento de Memória

O objetivo desta fase é preparar o Kernel para manipular a memória gráfica de forma eficiente e segura, sem tocar ou reescrever a VRAM real diretamente a cada comando de desenho.

Milestone 1.1: Configuração do Modo de Vídeo Linear

- ☐ Configurar o **UEFI GOP (Graphics Output Protocol)** ou **VESA VBE** no Bootloader para obter o endereço base do Framebuffer Linear.
- ☐ Mapear o endereço físico do Framebuffer na tabela de páginas do Kernel (*Paging*) utilizando o atributo de cache **Write-Combining** para máxima performance de escrita de dados da CPU para a GPU.
- ☐ Criar uma estrutura de dados global no Kernel para o controle do modo de vídeo:

```
struct VideoMode {
    uint32_t* vram_address;
    uint32_t width;
    uint32_t height;
    uint32_t pitch; // Bytes por linha física na tela
    uint8_t bpp;    // Bits por pixel (use 32-bit para ARGB)
};
```

Milestone 1.2: Subsistema de Alocação Dinâmica (MALLOC)

- ☐ Garantir que o gerenciador de memória virtual do Kernel (*Virtual Memory Manager*) consiga alocar blocos de memória contíguos e grandes na RAM (na escala de vários Megabytes).
- ☐ Implementar funções estáveis de `malloc()` e `free()` no espaço do kernel. Cada nova janela instanciada e imagem carregada necessitará de seu próprio espaço alocado dinamicamente.

Milestone 1.3: O Backbuffer Principal (Double Buffering)

- ☐ Durante a inicialização do subsistema gráfico, aloque um bloco de memória na RAM comum exatamente do mesmo tamanho da tela ($Largura \times Altura \times 4$ bytes).
- ☐ Implementar a função otimizada `screen_swap()` : um laço altamente otimizado (utilizando instruções SSE/AVX caso o kernel dê suporte) que realiza o `memcpy` completo do Backbuffer diretamente para o endereço físico da VRAM.

Fase 2: O Motor de Blitting e o Gerenciador de Bitmaps

Nesta etapa, eliminam-se em definitivo os algoritmos de desenho manual por laço de pixel no código principal, substituindo-os pela renderização automatizada de blocos e imagens pré-fabricadas.

Milestone 2.1: Desenvolvimento da Engine de Blitting

- ☐ Implementar a função matemática `graphics_blit` , responsável por copiar matrizes retangulares de memória:

```
void graphics_blit(uint32_t* dest_buffer, int dest_pitch, int dest_x, int dest_y,
                  uint32_t* src_buffer, int src_w, int src_h, int src_pitch,
                  int src_x, int src_y, int width, int height);
```

- ☐ Adicionar suporte rígido a **Clip Rectangles**: a função deve truncar o desenho de forma automática caso o processo tente desenhar fora dos limites físicos da tela ou da janela, prevenindo corrupção de memória (*Kernel Panic*).

Milestone 2.2: O Pipeline de Assets (Ferramental Externo)

- ☐ Desenvolver um script utilitário (em Python ou C) para rodar na sua máquina de desenvolvimento principal (Host).
- ☐ O script deve ler arquivos de imagem comuns como `.bmp` ou `.tga` de 32 bits e exportá-los como arquivos de cabeçalho C (`.h`), contendo um array bruto estático de `uint32_t` estruturado em formato hexadecimal `0xAARRGGBB` .

Milestone 2.3: Renderização de Bitmaps no OS

- ☐ Embutir o papel de parede, os logotipos e os ícones básicos compilando os arquivos de cabeçalho gerados diretamente junto com a imagem do sistema operacional.
- ☐ Implementar suporte a **Alpha Blending** (Transparência real) na função de blit através da aplicação da fórmula de interpolação linear por pixel:

$$Cor_{Final} = (Cor_{Src} \times Alpha) + (Cor_{Dest} \times (1 - Alpha))$$

Fase 3: O Display Server (Servidor de Janelas)

Esta camada lógica abstrata gerencia o posicionamento, estados e o isolamento dos buffers de memória de cada aplicativo em execução.

Milestone 3.1: Abstração da Estrutura de Janelas

- ☐ Definir a estrutura de dados central que representará as janelas no sistema:

```
typedef struct Window {
    uint32_t id;
    int x, y;
    int width, height;
    uint32_t* buffer; // Framebuffer privado reservado desta janela
    uint32_t flags;    // Ex: TRANSPARENT, NO_BORDER, MOVABLE
    struct Window* next;
    struct Window* prev;
} window_t;
```

Milestone 3.2: Gerenciador de Árvore Visual (Z-Order)

- ☐ Implementar uma lista duplamente encadeada global para rastrear todas as janelas ativas.
- ☐ Escrever as funções fundamentais de controle de nós da lista: `window_create()` (alocação do buffer privado), `window_bring_to_front()` (mover o nó para o fim da lista, garantindo renderização no topo) e `window_destroy()`.

Milestone 3.3: O Driver de Mouse e Sistema de Input

- ☐ Consolidar o driver de interrupção do mouse (seja via barramento PS/2 clássico ou pacotes USB HID).
- ☐ O driver de interrupção deve unicamente atualizar as variáveis globais de coordenadas `mouse_x` e `mouse_y` a cada movimento, sem jamais realizar operações diretas de desenho na tela.

Fase 4: O Compositor Gráfico

O compositor atua como o laço de execução principal (Render Loop) do ambiente visual, unificando os buffers isolados, as imagens e as entradas em tempo real.

Milestone 4.1: O Laço do Compositor (Render Loop)

- ☐ Criar a rotina central do loop gráfico que executa a cada atualização de tela:
 - ☐ 1. Limpar o Backbuffer Principal executando o blit da imagem do papel de parede.
 - ☐ 2. Percorrer a lista encadeada de janelas do início ao fim (do plano de fundo para a frente).

- ☐ 3. Para cada nó, disparar o `graphics_blit` copiando os pixels de `window->buffer` para a posição correspondente no Backbuffer.
- ☐ 4. Desenhar o bitmap do cursor do mouse na posição atual das variáveis globais utilizando o algoritmo de Alpha Blending.
- ☐ 5. Invocar a função `screen_swap()` para atualizar o display de forma atômica.

Milestone 4.2: Decoração de Janelas Automatizada

- ☐ Configurar o Display Server para injetar de forma automática uma "moldura" padronizada (contendo barras de título e botões de fechamento através de bitmaps internos) ao redor do buffer de pixels bruto enviado pelo aplicativo.

Milestone 4.3: Interatividade (Movimentação e Foco)

- ☐ Implementar lógica de detecção de colisão: quando o botão do mouse for acionado, o compositor varre a lista de trás para frente para identificar qual janela interceptou o clique.
- ☐ Se o clique ocorrer na barra de título, transicione o estado da janela para "em arrasto" e atualize suas variáveis `x` e `y`. Invoque `window_bring_to_front()` para reorganizar o foco do Z-Order.

Fase 5: A API do Usuário (User-Mode Interface)

A etapa conclusiva na qual processos externos e isolados ganham a capacidade de renderizar telas completas utilizando a sua infraestrutura gráfica de maneira padronizada.

Milestone 5.1: Chamadas de Sistema (Syscalls) Gráficas

- ☐ Expor as interfaces e pontos de entrada de chamadas de sistema necessárias para os aplicativos:
 - ☐ `sys_create_window(w, h)`
 - ☐ `sys_refresh_window(window_id)`
 - ☐ `sys_get_window_buffer(window_id)` (retorna o endereço mapeado onde o aplicativo pode realizar suas escritas livremente).

Milestone 5.2: Isolamento e Estabilidade

- ☐ Garantir o isolamento de memória: caso um aplicativo falhe, sofra um travamento interno ou corrompa os dados contidos dentro do seu próprio `window->buffer`, o Compositor apenas lerá dados inválidos, mas o Kernel e as demais janelas permanecerão intactos e perfeitamente estáveis.